程序设计综合实验

实验 1：Git、C 指针与时间测量

黄韬
taohonker@gmail.com
昆明理工大学·系统科学系

2026-09-16

参考与致谢

讲义内容（包括文字、图片、讲解思路等）编撰时大量借鉴如下参考资料：

- GeeksForGeeks 网站的众多教程
- 《Hello 算法》开源全彩算法入门

在此一并致谢，不再一一引用。

其它引用见正文内容。

实验简介与要求

这个实验主要是为即将开始的数据结构与算法分析课程做准备，包括如下内容：

1. 复习 Git 的使用方法
2. 复习 C 语言指针的使用方法与相关知识
3. 学习如何在 C 和 Python 中测量时间，并计算常见操作符和简单函数的时间消耗

实验要求：

- 提交形式：实验以 markdown + 代码的形式提交，不使用 Pull Request。
- 提交流程：每位同学在自己的仓库独立完成并提交，保留完整的 Git 提交记录，推送到自己的 fork 仓库。
- 提交时间：下周三（26-09-23）晚 23:59 前完成推送。

Git 简介

参考资料

1. Nulab Git 教程，简单直白，概念清晰
2. 廖雪峰的 Git 教程，强调上手操作，快速入门
3. Pro Git 第 2 版中文版，非常专业的书籍

关于 Git

Git 的定义

- **Git**：一种分布式版本控制系统，用于跟踪文件的更改。
- **版本控制**：记录文件随时间的变化，允许回溯到以前的版本。

为什么使用 Git?

- **协作**：多个开发者可以同时工作，不会互相干扰。
- **历史记录**：可以查看项目的历史记录，了解每次更改的原因和内容。
- **分支管理**：可以创建分支，独立开发新功能或修复 bug，然后合并回主分支。

Git 交友

世界上众多的程序员（可能超过 90%）在使用 git 管理他们的文件，其中很多集中在

- Github.com
- GitLab.com

Git 的基本概念

工作目录（Working Directory）

- **定义**：你正在进行更改的文件所在的目录。
- **作用**：保存当前项目的状态。

暂存区（Staging Area）

- **定义**：准备提交更改的区域。
- **作用**：允许你在提交前检查和调整更改。

本地仓库（Local Repository）

- **定义**：存储在你计算机上的项目历史记录。
- **作用**：包含所有的提交和分支，是项目的个人记录。

远程仓库（Remote Repository）

- **定义**：托管在网络上的项目版本。
- **作用**：允许多个开发者协作，通过推送和拉取共享资源。

Git 的基本概念

分支（Branches）

- **定义**：项目的并行版本。
- **作用**：允许独立开发不同的功能或修复，直到准备好合并回主项目。

合并（Merge）

- **定义**：将一个分支的更改整合到另一个分支。
- **作用**：结合两个分支的历史记录，创建一个统一的历史。

Git 的安装和配置

安装 Git

- **Windows**：下载并运行 Git for Windows 安装程序。
- **macOS**：使用 Homebrew 安装：`brew install git`。
- **Linux**：使用包管理器安装，例如 Ubuntu：`sudo apt install git`。

配置 Git

- **设置用户名和邮箱**：

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

- **查看配置**：

```
git config --list
```

Git 的基本操作

实验内容

完成下面每一项任务，并记录实验过程与结果。提出的问题尽量回答并展示解决过程。

创建和克隆仓库

创建本地仓库

- **命令**：`git init`
  - **作用**：在当前目录初始化一个新的 Git 仓库。
  - **示例**：

```
mkdir my_project
cd my_project
git init
```

克隆远程仓库

- **命令**：`git clone <repository_url>`
  - **作用**：从远程仓库复制一个完整的仓库到本地。
  - **示例**：

```
git clone https://github.com/user/repo.git
```

提交更改

添加文件到暂存区

- **命令**：`git add <file>`
  - **作用**：将文件添加到暂存区，准备提交。
  - **示例**：

```
# 暂存某文件
git add file.txt
# 暂存当前目录所有文件
git add .
```

- 问题：如何撤销暂存？

提交更改

- **命令**：`git commit -m "Commit message"`
  - **作用**：将暂存区的更改提交到本地仓库。
  - **示例**：

```
git commit -m "Initial commit"
```

- 问题：如何撤销提交？

查看状态和历史

查看仓库状态

- **命令**：`git status`
  - **作用**：显示工作目录和暂存区的状态。

查看提交历史

- **命令**：`git log`
  - **作用**：显示提交历史记录。

Git 的分支管理

注意：每一个操作中都是在某个分支的某个状态节点上进行，所以要想清楚每一个命令中没有显式写出的**当前分支与节点**。

创建新分支

- **命令**：`git branch <branch_name>`
  - **作用**：创建一个新的分支。

切换分支

- **命令**：`git checkout <branch_name>`
  - **作用**：切换到指定的分支。

合并分支到当前分支

- **命令**：`git merge <branch_name>`
  - **作用**：将指定分支的更改合并到当前分支。
  - **示例**：

```
# 切换到 main 分支
git checkout main
# 将 feature-branch 分支合并到 main 分支
git merge feature-branch
```

Git 的远程操作

添加远程仓库

- **命令**：`git remote add <name> <url>`
  - **作用**：添加一个新的远程仓库。

查看远程仓库

- **命令**：`git remote -v`
  - **作用**：显示所有远程仓库的详细信息。

推送到远程仓库

- **命令**：`git push <remote> <branch>`
  - **作用**：将本地分支的更改推送到远程仓库。

从远程仓库拉取

- **命令**：`git pull <remote> <branch>`
  - **作用**：从远程仓库拉取并合并更改到本地分支。
- 问题：如果出现冲突一般是什么原因？怎么解决冲突？

实验内容：理解 C 语言中的指针

参考资料

- 《“笨办法”学 C 编程》，课余时间坚持看完，有收益
- 《C 指针》
- 《C 语言的 495 个问题》

指针变量可以指向自身吗?

- **问题**：指针变量可以指向自身吗？如果可以，如何定义和初始化这样的变量？
- **解释**：指针可以指向自身，但这并不常见且通常不实用，只作为加深理解指针的基础。
- 可能遇到的麻烦：指针指向自身时，被指类型与指针类型不匹配，可能导致错误。
- 可能的方案：
  - 使用 `void*` 指针
  - 使用 `uintptr_t` 指针
  - 使用函数指针
  - 使用 union
- **提示**：使用 `printf` 中的 `%p` 格式说明符打印指针存储的地址。

指针的值

- **问题：**
  1. 指针的值是什么样的？打印它。
  2. 指针占用多少内存？
  3. 如果指针指向 `short` 或 `double`，它的大小会不同吗？
- 给出程序来解答
- **提示：**使用 `printf` 中的 `%p` 格式说明符打印指针存储的地址。

指针、数组、指针数组

- **任务**：定义并操作如下内容
  1. 指向指针的指针
  2. 二维数组
  3. 指针数组
  4. 指向数组的指针
- 比较它们，有什么联系和区别？数组和指针之间有什么联系和区别？

改变指针的值

- **问题：**
  - 如何改变指针的值？有几种方法？
  - 如何对指针的原始值加一个整数偏移？后果是什么？

数组与指针

- 数组在内存中的存在形式是什么?
  - 数组在内存中是以连续的内存块存储的。
  - 每个元素在内存中的地址是连续的。
- 如何用指针操作数组?
- 如何用指针方式为函数传入数组?

数组的维度和形状

- 定义多维数组，例如二维数组、三维数组等。
- 用指针操作高维数组

在 Python 里有没有指针？有没有指针的替代品？如何使用？

- Python 中没有传统意义上的指针。
- Python 使用引用来管理对象。

指针的替代品：可以使用列表、字典等数据结构来模拟指针的行为

```
def modify_list(lst):
    lst[0] = 100
my_list = [1, 2, 3, 4, 5]
modify_list(my_list)
print(my_list)  # 输出: [100, 2, 3, 4, 5]
```

使用引用：Python 中的变量实际上是对对象的引用

```
a = [1, 2, 3]
b = a
b[0] = 100
print(a)  # 输出: [100, 2, 3]
```

延伸：Python 中的对象与引用

在 Python 中，改变变量的值的行为取决于变量的数据类型。Python 中的变量实际上是对对象的引用，而不是直接存储值。以下是详细的解释：

不可变数据类型

对于不可变数据类型（如整数、浮点数、字符串和元组），改变变量的值会创建一个新的对象，并将变量引用指向这个新对象。

示例：

```
a = 5
print(id(a))  # 输出对象 a 的内存地址

a = 10
print(id(a))  # 输出对象 a 的新内存地址
```

在这个例子中，a 最初指向值为 5 的对象。当 a 被赋值为 10 时，Python 创建了一个新的对象，并将 a 的引用指向这个新对象。

延伸：Python 中的对象与引用

可变数据类型

对于可变数据类型（如列表、字典和集合），改变变量的值通常是在原有对象上进行修改，而不会创建新的对象。

示例：

```
lst = [1, 2, 3]
print(id(lst))  # 输出列表 lst 的内存地址

lst.append(4)
print(id(lst))  # 输出列表 lst 的内存地址，地址不变
```

在这个例子中，`lst` 是一个列表。当我们使用 `append` 方法向列表中添加元素时，列表的内存地址没有改变，说明是在原有对象上进行的修改。

总结

- **不可变数据类型**：改变值会创建一个新对象，并改变引用。
- **可变数据类型**：改变值通常是在原有对象上进行修改，不会改变引用。

在 C 和 Python 中计时代码执行时间

实验内容

1. 学习如何在 C 和 Python 中计时代码执行时间。
2. 测量常见操作符和简单函数（如 `+`、`-`、`*`、`/`、`sin`、`log`、`exp`、`power` 等）的时间消耗。

在 C 中计时代码执行时间

使用 `clock()` 函数，在代码开始与结束处调用两次 `clock()` 函数，计算两次调用之间的时间差，并除以 `CLOCKS_PER_SEC` 得到秒数。

在 Python 中计时代码执行时间

1. 使用 `time` 模块，在代码开始与结束处分别调用两次 `time.time()` 函数，计算两次调用之间的时间差。
2. 使用 `timeit.timeit()` 函数计时代码执行时间。

测量常见操作符和函数的时间消耗

使用上一小节的计时方法，分别测量加法、减法、乘法、除法以及 `sin`、`log`、`exp`、`power` 等函数的单次执行时间，记录并比较不同操作的时间量级。